

Informe del Proyecto — Unidad 2

Autor: Vejat Olea

Fecha: Julio 2026

Introducción

Para este proyecto desarrollamos un keylogger educativo en Python, probado únicamente en máquinas virtuales propias dentro de VirtualBox. La idea fue simular un escenario de espionaje entre una máquina víctima y una máquina atacante, siempre en un entorno controlado y sin utilizar equipos de terceros.

El trabajo se organizó en cuatro ejercicios: implementar el keylogger con persistencia, cifrar y enviar los datos capturados, compilar el programa y analizar su detección, y redactar un informe técnico de amenaza. En este documento explico cómo desarrollamos cada parte, qué resultado obtuvimos y por qué consideramos que cumple con lo solicitado.

Entorno de trabajo

Montamos dos máquinas virtuales con Windows 11: una víctima (192.168.56.107) y otra atacante (192.168.56.108), conectadas mediante red Host-Only para que la comunicación quedara dentro del laboratorio. Además, compartimos el código del proyecto con una carpeta de VirtualBox disponible en ambas VMs, lo que nos permitió probar el programa en los dos lados y compartir el archivo `secret.key` entre el keylogger y el receptor sin tener que copiar archivos manualmente.

Ejercicio 1 — Keylogger y persistencia

En el primer ejercicio desarrollamos el núcleo del keylogger. El programa captura las teclas del usuario con `pynput`, las guarda en memoria y cada veinte segundos las envía al servidor del atacante, donde un receptor Flask las recibe y las muestra. Para verificar su funcionamiento, escribimos en el Bloc de notas y comprobamos que las teclas llegaban correctamente al otro equipo, demostrando así cómo un atacante puede espiar la escritura de un usuario sin que este lo note.

A continuación implementamos persistencia, ya que un implante de este tipo debería seguir activo después de reiniciar Windows. El programa se registró en la clave Run de HKCU y dejó una copia en la carpeta de Inicio bajo el nombre `WindowsSecurityUpdateService`, simulando una actualización legítima. Tras reiniciar la VM víctima, el proceso volvió a ejecutarse solo y el receptor continuó recibiendo datos.

Sin embargo, al principio la persistencia no funcionaba porque el registro apuntaba a un alias de Python de la Microsoft Store que no arranca con Windows. Finalmente lo resolvimos con un archivo `.bat` que usa la ruta real de `pythonw.exe` y espera a que la carpeta compartida esté montada. También documentamos las limitaciones del keylogger: no captura bien contraseñas en navegadores, teclas en ventanas con UAC elevado ni texto pegado con Ctrl+V, lo que nos ayudó a entender que incluso un malware de este tipo tiene restricciones en la práctica.

Ejercicio 2 — Cifrado y envío de datos

En el segundo ejercicio protegimos la información antes de enviarla por la red, cifrándola con AES-256-GCM. La clave se genera en la primera ejecución y se guarda en `secret.key`, mientras que el receptor descifra con ese mismo archivo. Elegimos cifrado simétrico en lugar de un hash porque necesitábamos recuperar el texto original; un algoritmo como SHA no permitiría mostrar las teclas capturadas.

Durante las pruebas vimos que en el receptor aparecía el texto legible, pero en la red solo se observaba un campo `payload` en base64 sin palabras reconocibles. Además, si el mensaje se altera o se usa una clave incorrecta, el descifrado falla con `InvalidTag`, lo que demuestra que AES-GCM también protege la integridad del dato y no solo su confidencialidad. De esta manera el cifrado de aplicación cumple su rol aunque la comunicación viaje por HTTP sin TLS, tal como ocurre en nuestro laboratorio.

Ejercicio 3 — Ejecutable, MITM y VirusTotal

El tercer ejercicio exigía entregar un binario funcional, por lo que compilamos el keylogger como `WindowsSecurityUpdateService.exe` con PyInstaller. De esta forma simulamos la distribución de un malware disfrazado de actualización del sistema. El ejecutable pesa alrededor de 14,4 MB y su hash SHA-256 quedó registrado como indicador de compromiso:

`7CEFCF86B89F2323E5279BF7D24D5BD424F4A52A6E831FE6CD6FDBF838E5F421`.

También realizamos una demostración de ataque en medio con Wireshark. Primero intentamos capturar desde el host, pero no apareció tráfico porque en VirtualBox la comunicación entre dos VMs no pasa por la tarjeta del host. Entonces capturamos directamente en la VM atacante y observamos envíos HTTP periódicos al puerto 5000. Aunque el tráfico era visible, el `payload` viajaba cifrado, de modo que un interceptador no podía leer las teclas sin `secret.key`.

Por último, analizamos el ejecutable en VirusTotal y obtuvimos 19 detecciones de 69 motores (27,5%), con la etiqueta `trojan.clyp/keylogger`. ESET y Kaspersky lo identificaron como keylogger Python, mientras que Microsoft y Avast no lo marcaron. En consecuencia, el programa logra evasión parcial, pero no total.

Ejercicio 4 — Informe técnico de amenaza

En el cuarto ejercicio redactamos un informe de amenaza con formato Threat Intelligence, describiendo nuestra herramienta como un implante real. Incluimos la descripción del malware, el vector de infección, las TTPs según MITRE ATT&CK, los indicadores de compromiso, el impacto, las medidas de mitigación y las conclusiones. También registramos datos concretos como nombres de archivos, rutas de registro, direcciones de red y hashes del ejecutable, porque un informe de este tipo debe servir para que un equipo de seguridad pueda buscar la infección en sus sistemas. En definitiva, este ejercicio nos obligó a analizar el proyecto desde la perspectiva defensiva y no solo desde la del atacante. El informe completo está en `docs/informe_tecnico.md`.

Conclusión

En conjunto, construimos un keylogger que captura teclas, persiste tras reinicio, cifra su comunicación con AES-256-GCM y exfiltra datos cada veinte segundos hacia un servidor del atacante. Lo probamos en

laboratorio con dos VMs, lo compilamos como ejecutable y medimos su detección en VirusTotal.

Lo más relevante que aprendimos es que un atacante puede combinar persistencia, sigilo, empaquetado y cifrado para dificultar la detección, pero ninguna capa es perfecta. El tráfico sigue siendo visible en la red, algunos antivirus detectan el binario y el keylogger no captura todo lo que hace el usuario. Por eso la defensa también debe ser en capas: revisar inicios automáticos, usar EDR, analizar patrones de red y promover buenas prácticas entre los usuarios.

Todo el desarrollo y las pruebas se realizaron exclusivamente en máquinas virtuales propias, con fines educativos.